GESTURE DRIVEN IoT SMART HOME SYSTEM

19EEPN6601 - INNOVATIVE AND CREATIVE PROJECT

Submitted by

HARINI R K	**727622BEE054**
ARULSELVAN M	**727622BEE104**
THARUN RAJ R	**727623BEE302**

in partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

in

ELECTRICAL AND ELECTRONICS ENGINEERING

Dr. MAHALINGAM COLLEGE OF ENGINEERING AND TECHNOLOGY POLLACHI - 642003

An Autonomous Institution

Affiliated to Anna University, Chennai - 600 025

MAY 2024

Dr. Mahalingam College of Engineering and Technology

Pollachi-642 003 An Autonomous Institution

Affiliated to Anna University, Chennai-600 025

BONAFIDE CERTIFICATE

Certified that this project report,

"Gesture Driven IoT Smart Home System"

is the bona-fide work of

HARINI R K	**727622BEE054**
ARULSELVAN M	**727622BEE104**
THARUN RAJ R	**727623BEE302**

who carried out the project work under my supervision

SIGNATURE OF HOD	**SIGNATURE OF SUPERVISOR**
Dr. D. GANESHKUMAR , M.E., Ph.D.,	**Dr. K.VIJAYAKUMAR, M.E., Ph.D.,**
HEAD OF THE DEPARTMENT, Electrical and Electronics Engineering Dr. Mahalingam College of Engineering and Technology, NPT-MCET Campus Pollachi-642003.	Associate Dean (Academics), Electrical and Electronics Engineering Dr. Mahalingam College of Engineering and Technology, NPT-MCET Campus Pollachi-642003.

Submitted for the Autonomous End Semester Examination Mini Project Viva-Voce held on _____

INTERNAL EXAMINER	EXTERNAL EXAMINER

Dr. Mahalingam College of Engineering and Technology
Pollachi -642003

Technology Readiness Level (TRL) Certificate

Gesture Driven IoT Smart Home System
Course Code: 19EEPN6601

Students Names and Roll Numbers:

HARINI R K	727622BEE054
ARULSELVAN M	727622BEE104
THARUN RAJ R	727623BEE302

Guide Name: Mr. K. VIJAYAKUMAR Associate Dean /EEE

Technology Readiness Level* (TRL) of this Project: _____

Signature of the Guide HoD

Internal Examiner External Examiner

ACKNOWLEDGEMENT

We extend our gratitude to our management for having provided us with all facilities to build our project successfully. We express our sincere gratitude to our honourable Secretary **Dr. C. Ramaswamy, M.E., Ph.D., F.I.V.,** for providing uswith required amenities.

We express our profound thanks to our principal **Dr. P. Govindasamy, B.E., M.E., Ph.D.,** who provided us the suitable environment to work.

We express our gratefulness to **Dr. A. Senthilkumar, M.E., Ph.D.,** Dean Academic & Autonomous for providing us kind advice during the development of the project.

We express our gratefulness to **Dr. D. GANESHKUMAR, M.E.., Ph.D.,** Professorand Head of the Department, Electrical and Electronics Engineering for providing us kind advice during the development of the project.

Our hearty thanks to our guide **Mr. K. VIJAYAKUMAR, M.E., Ph.D.,** Associate Dean, Electrical and Electronics Engineering for his constant support and guidance offered to us during the course of our project by being one among us.

We are committed to place our heartfelt thanks to all our faculty members, lab technicians and friends, who played the supporting role throughout the project.

1
2

LIST OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	**vi**
	LIST OF FIGURES	**vii**
	LIST OF ABBREVIATIONS	**viii**
1	**INTRODUCTION**	
	1.1 INTRODUCTION	1
	1.2 LITERATURE REVIEW	3
	1.2.1 Solar Power Monitoring System Using IoT	3
	1.2.2 Enhancing Solar Power Generation Through Real-Time Monitoring of Current and Voltage	4
	1.2.3 IoT Based Solar Energy Monitoring System	6
	1.2.4 Real Time Online Monitoring of Solar Power Plants	7
	1.2.5 IoT Based Intelligent Smart Energy Monitoring System for Solar PV Power Generation	8
	1.2.6 Monitoring System Solar Power Utilizing IoT System	10
	1.2.7 Smart Grid Monitoring and Control via Internet of Things	11
2	**PROPOSED SYSTEM**	**13**
	2.1 BLOCK DIAGRAM	13
	2.2 FLOW CHART	14
	2.3 SYSTEM SPECIFICATIONS	15
	2.3.1 Voltage Sensor	15
	2.3.2 Arduino Uno	16
	2.3.3 Current Sensor	17
	2.3.4 LCD Module	18
	2.3.5 Solar Panel	19
	2.3.6 I2C Interface Module	20
	2.3.7 ESP32 Module	21

3	**SOFTWARE IMPLEMENTATION**	**22**
	3.1 SIMULATION CIRCUIT	22
	3.2 SIMULATION OUTPUT	23
	3.3 TYPE OF SENSOR NEEDED FOR IOT	24
	3.4 ALOGORITHMS	25
4	**RESULTS AND DISCUSSIONS**	**26**
	4.1 HARDWARE OUTPUT	26
5	**CONCLUSION**	**27**
	REFERENCES	**28**
	APPENDIX	**29**
	PLAGIARISM REPORT	**36**

ABSTRACT

This project introduces a Gesture Driven IoT Smart Home System designed to provide a hygienic and contactless solution for controlling four household appliances using simple hand gestures. Leveraging a webcam and advanced computer vision techniques through Python libraries such as OpenCV, Media-Pipe, and a custom Hand Tracking Module, the system detects and interprets hand gestures in real-time. Media-Pipe's hand landmark detection identifies finger positions, which are processed by a custom Hand Tracker class to recognize specific gestures. These gestures are then mapped to control commands and sent via serial communication to an Arduino UNO, which operates a relay module to switch appliances ON or OFF. This IoT-enabled solution promotes hygiene, reduces contamination risks from physical switches, and offers a safe, convenient, and intuitive method for smart home interaction.

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
1.1	Solar monitoring System	3
1.2	Block Diagram of System	4
1.3	Block Diagram of System	6
1.4	Block Diagram of System	8
1.5	Block Diagram of System	9
1.6	Block Diagram of System	10
1.7	Block Diagram of System	11
1.8	Solar Power Plants	12
2.1	Block Diagram of Solar Monitoring System	13
2.2	Flowchart Of Solar Monitoring System	14
2.3	Voltage Sensor	15
2.4	Arduino UNO	16
2.5	Current Sensor	17
2.6	LCD Module	18
2.7	Solar Panel	19
2.8	I2C Interface Module	20
2.9	ESP32 Module	21
3.1	Simulation Circuit	22
3.2	Simulation Output	23
3.3	Types of Sensors needed	24
4.1	Solar Power Monitoring System	26
4.2	Hardware Output	26

LIST OF ABBREVATIONS

TERMS	ABBREVIATIONS
IoT	Internet of Things
LCD	Liquid Crystal Display
UDM	Underground Drainage Monitoring
IDE	Integrated Development Environment
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheet

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION:

In recent years, the concept of smart homes has advanced significantly, with systems becoming more intelligent, connected, and focused on enhancing user convenience. Traditionally, these systems required physical interaction through manual switches or mobile applications. While effective, these methods demand direct contact, which raises hygiene concerns—especially in the wake of global health crises like the COVID-19 pandemic. This has accelerated the demand for contactless solutions that ensure both ease of use and improved sanitary conditions. One such promising alternative is gesture-based control, which enables users to interact with appliances through simple hand movements, offering a natural, intuitive, and hygienic interface without the need for physical touch.

Gesture recognition technology interprets human hand motions using advanced computer vision and machine learning techniques. Unlike voice commands, which can be affected by ambient noise or require clear enunciation, gestures offer a silent and visually-driven mode of communication with devices. They are especially useful in environments where users may have wet or dirty hands, are wearing gloves, or simply prefer non-verbal interaction. This hands-free capability is particularly beneficial for individuals with physical disabilities or mobility issues, making gesture-based interfaces a more inclusive option for smart home control.

The Gesture Driven IoT Smart Home System proposed in this project leverages these advantages by allowing users to control up to four electrical appliances using distinct hand gestures. At the heart of this system is a webcam, which continuously captures video in real time. The video feed is processed using Python libraries such as OpenCV and Media-Pipe, along with a custom-built Hand Tracking Module that accurately detects hand landmarks and interprets finger positions. The core of this system is a gesture recognition algorithm embedded within a custom Hand Tracker class that identifies which fingers are raised and maps them to specific commands.

These interpreted gestures are then translated into control instructions and transmitted via serial communication to an Arduino UNO microcontroller. The Arduino is programmed to receive these commands and activate a relay module accordingly, turning connected appliances ON or OFF. This setup enables smooth and responsive control of home electronics without requiring physical interaction. The use of serial communication ensures reliable data transmission between the computer and Arduino, bridging the software-driven gesture recognition system with hardware-based appliance control.

By integrating gesture recognition with IoT and embedded systems, the project highlights a seamless blend of software and hardware that enhances the functionality of modern homes. The system is designed to be scalable, adaptable to various devices, and customizable to suit different user preferences or gesture sets. Furthermore, it operates efficiently using readily available and low-cost components, making it an accessible solution for widespread adoption in residential or even commercial spaces.

The Gesture Driven IoT Smart Home System offers a forward-thinking, hygienic, and user-friendly approach to home automation. It addresses the growing need for contactless interaction while embracing the natural human tendency to use gestures for communication. This project not only advances the capabilities of smart home environments but also emphasizes the importance of safety, accessibility, and intuitive design in the future of home automation technologies.

1.2 LITERATURE REVIEW

1.2.1 Suresh Babu et al [2024] have proposed the system *Smart Gesture Controlled Systems Using IoT* to revolutionize interaction with household appliances by eliminating the need for physical contact. The system uses advanced sensor technologies like motion and infrared sensors to detect and interpret hand gestures such as waving or swiping. These gestures are converted into commands for controlling devices like lights, fans, and switches. It also uses artificial intelligence and machine learning to enhance gesture recognition accuracy over time. Users install gesture-controlled switches and connect them to a smart hub or smartphone app, enabling gesture customization and remote control. This IoT-enabled system promotes convenience, hygiene, and energy efficiency in smart home environments. [1]

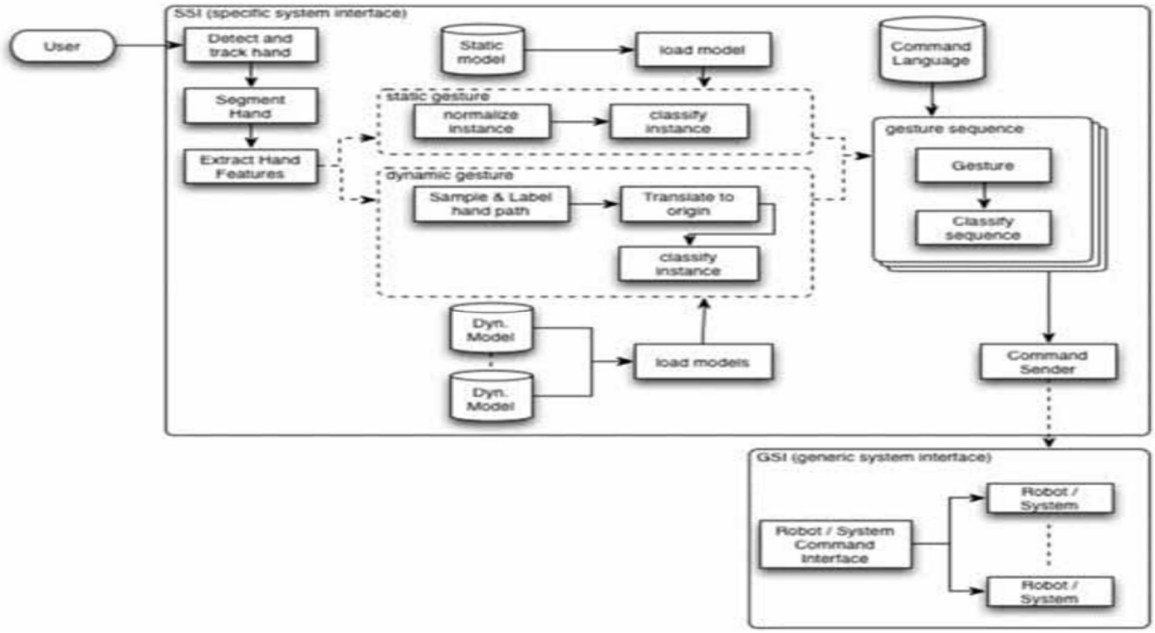


Figure 1.1 Block Diagram of solar power monitoring system using IOT

Smart gesture-controlled systems are an emerging technology that allow users to interact with devices through hand or body movements. These systems translate specific gestures into actionable commands using a combination of sensors, machine learning algorithms, and software interfaces. This innovation has the potential to transform human-technology interaction, making it more accessible and user-friendly. Although gesture recognition has long been used in human-computer interaction, recent advancements in sensor technology and machine learning have significantly improved the accuracy and responsiveness of these systems.

1.2.2 Fatima el.al [2022] have proposed the system *IoT-based Smart Home Automation Using Gesture Control and Machine Learning for Individuals with Auditory Challenges* to enhance accessibility for the deaf and hard-of-hearing community. This system uses a video-based American Sign Language (ASL) recognition approach that integrates MediaPipe for hand tracking, OpenCV for image normalization, and a Convolutional Neural Network (CNN) for gesture classification. Developed in Python, it processes real-time video streams to detect and interpret ASL gestures with high accuracy, enabling intuitive interaction with household devices. The system aims to bridge communication gaps and promote inclusivity by enabling gesture-based control of smart home appliances via Arduino UNO and IoT. By combining computer vision and machine learning, the framework addresses limitations of traditional gesture systems, such as sensitivity to lighting and low diversity in training data. Additionally, it eliminates the need for wearable devices like data gloves, offering a cost-effective and non-invasive solution. This approach not only improves user-device interaction but also supports applications in accessibility, elderly care, and emergency response, contributing to a more inclusive and sustainable smart home environment. Future developments may further enhance system precision and expand its applicability across diverse real-world scenarios.[2]

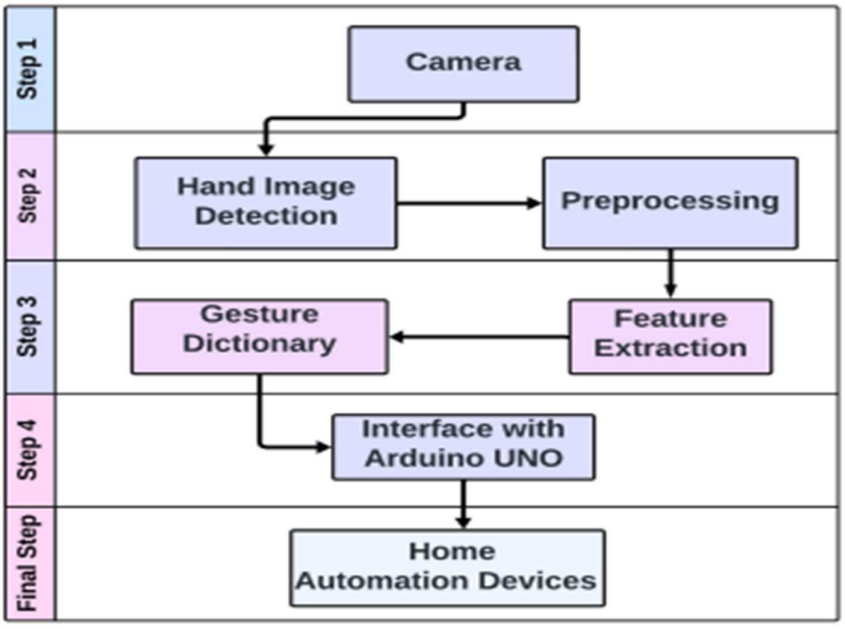


Figure 1.2 Block diagram of Gesture Recognition and Automation Execution

Gestures are a vital form of communication, and their absence can create challenges for individuals in everyday activities. To address this, a gesture recognition system has been developed using OpenCV, Python, and the MediaPipe framework. This system captures real-time hand gestures through a webcam, processes them immediately, and translates them into control commands for home appliances like lights and fans, with the help of an Arduino UNO module that manages the devices.

While vision-based hand tracking systems are widely used, they have limitations, such as restricted movement detection and difficulty handling overlapping or partially hidden hands. To overcome these challenges, the proposed system uses color and depth data to segment the hand, isolating the palm, wrist, and fingers while disregarding unnecessary parts like the forearm. Key hand gestures are recognized by analyzing finger positions, heights, and the distance from the palm's center

The gesture recognition process begins by capturing frames from a video feed. These frames are then processed in multiple stages: tracking, segmentation, feature extraction, and classification. During the tracking phase, the system analyzes the spatial and temporal movement of the hand, then isolates the gesture and extracts relevant features such as shape and motion. These features are classified according to predefined algorithms to determine the corresponding gesture

Hand detection starts by capturing the hand from the camera feed, followed by pre-processing to refine the image. Feature extraction and gesture mapping are performed next, correlating the features to a gesture dictionary for accurate identification. Once a gesture is recognized, it is translated into a control action via the Arduino UNO board using the 'pyfirmata' library, enabling real-time device control.

The Hand Landmarks model estimates 21 key points on the hand, including the fingers, knuckles, and palm center, allowing precise tracking of even small movements. In live video streaming, the system uses tracked palms from previous frames to minimize processing, making the gesture detection more efficient. Using Media-Pipe's bounding box approach enhances tracking and reduces unnecessary palm detections, while a neural network architecture processes hand gestures at multiple stages for high-accuracy recognition.

1.2.3 Fatima el.al [2023] have proposed the system IoT-based Smart Home Automation Using Gesture Control and Machine Learning for Individuals with Auditory Challenges to improve accessibility for the deaf and hard-of-hearing community. The system leverages a video-based American Sign Language (ASL) recognition framework that integrates Media-Pipe for hand tracking, OpenCV for image normalization, and a Convolutional Neural Network (CNN) for gesture classification. Built using Python, it processes real-time video streams to accurately detect and interpret ASL gestures, facilitating intuitive control of smart home devices. This system aims to bridge communication barriers and foster inclusivity by enabling gesture-based control of household appliances through Arduino UNO and IoT technology. The combination of computer vision and machine learning addresses key limitations of traditional gesture systems, including sensitivity to lighting conditions and insufficient training data diversity. [3]

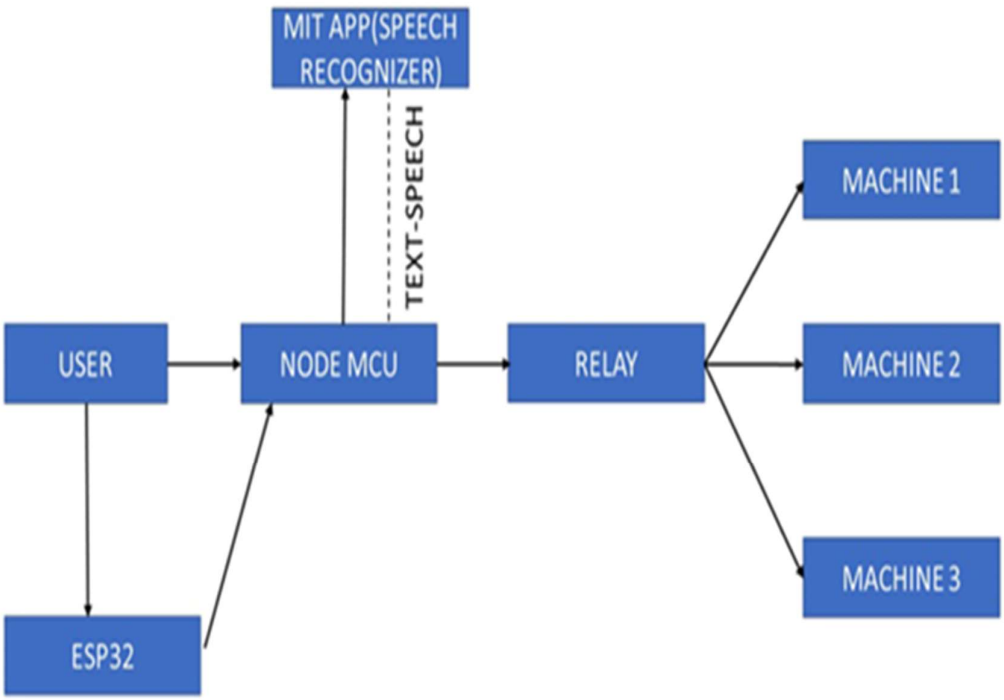


Figure 1.3 Block diagram of the system

Furthermore, it eliminates the need for wearable devices like data gloves, offering a cost-effective and non-invasive solution. In addition to enhancing user-device interaction, this approach has applications in accessibility, elderly care, and emergency response, contributing to a more inclusive and sustainable smart home environment. Future improvements are expected to further refine the system's accuracy and broaden its real-world applications.

1.2.4 Nazik Alturki el.al [2018] have presented an innovative approach in their paper titled Efficient and Secure IoT-Based Smart Home Automation Using Multi-Model Learning and Blockchain Technology to address key challenges in smart home environments. The system tackles issues like identity theft, data breaches, device security, and automated decision-making through a multi-model learning architecture. Specifically, it employs an ensemble of Random Forest (RF) and Convolutional Neural Networks (CNN) to classify devices as “ON” or “OFF” based on typical user behavior patterns. Furthermore, blockchain technology is integrated to ensure secure, decentralized authentication and recognition of IoT devices, enhancing trust and data integrity. The system utilizes affordable hardware such as a 5V relay, sensors, and a Raspberry Pi, with device control managed through a custom Android application communicating over an HTTP-based Apache server. The paper emphasizes the need for scalable, low-cost, and secure smart home frameworks and includes practical evaluations in both lab and real-world scenarios. The results affirm the system’s reliability and highlight its applicability in daily life for secure and intelligent home automation. [4]

This study presents a smart home automation system that enables users to remotely monitor and control appliances like lights, air conditioners, and sensors through a user-friendly graphical interface. Designed for energy efficiency and cost-effectiveness, the system is applicable in homes, hotels, and industrial settings. Users can customize their home layout via a drag-and-drop admin panel, which syncs data in JSON format with the server using VOLLEY. Real-time updates and notifications are provided, with the interface displaying device statuses, history, and user activity. A supervised learning classifier categorizes devices as “ON” or “OFF” using input features like device ID, room ID, temperature, and usage patterns. The most effective predictive model is chosen through comparative analysis. Additionally, blockchain technology secures communication and ensures authenticated data transmission between IoT devices and users, enhancing system reliability and privacy.

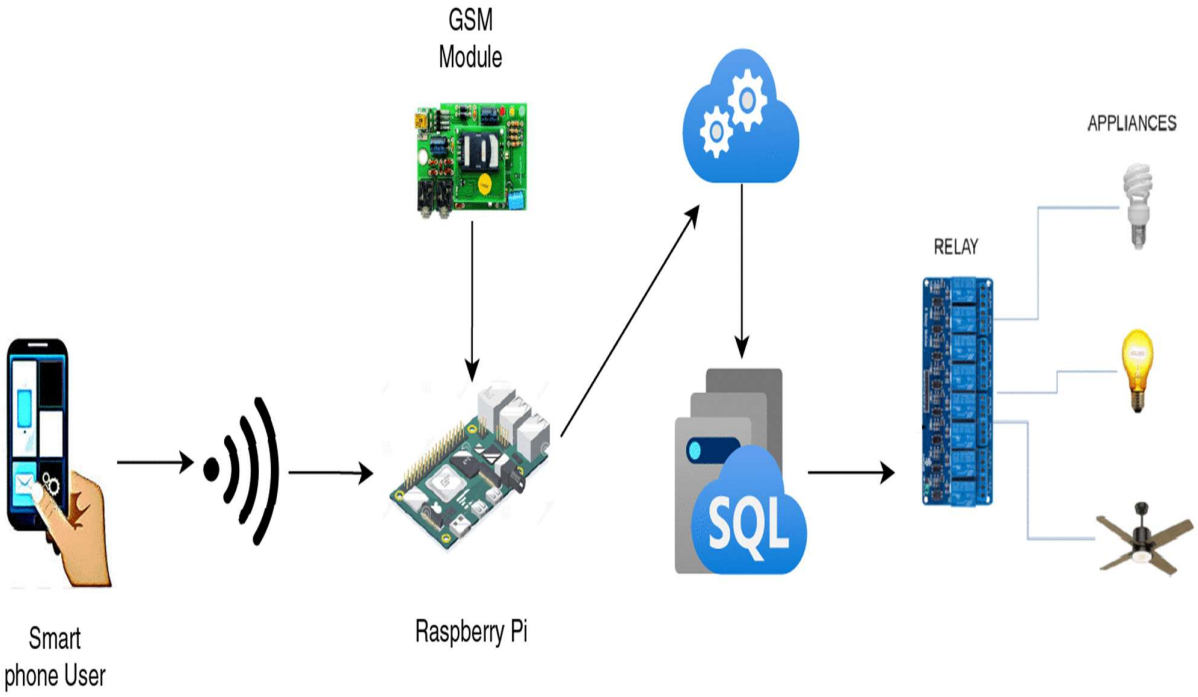


Figure 1.4 Block diagram of the system

1.2.5 Praveen Kumar el.al [2022] in his project *Gesture Based Home Automation System*, introduced a smart home solution that utilizes gesture control to operate household appliances, eliminating the need for physical touch or traditional remote controls. The system is built on an Arduino UNO and incorporates an APDS-9960 gesture sensor, known for its ambient light detection, proximity sensing, and directional gesture recognition capabilities. Users can navigate and control up to four appliances (demonstrated with 15-watt bulbs) by swiping their fingers in different directions—left/right to select the appliance and up/down to toggle its state. An LCD module displays the current appliance selection, and the relay circuit enables the physical switching of devices. The sensor communicates with the Arduino via an I2C interface, and open-source libraries are used for programming. Code development and uploading are performed using the Arduino IDE and AVR Dude. This project provides a low-cost, intuitive alternative to traditional home automation systems, particularly suited for individuals seeking hands-free control of their living environment

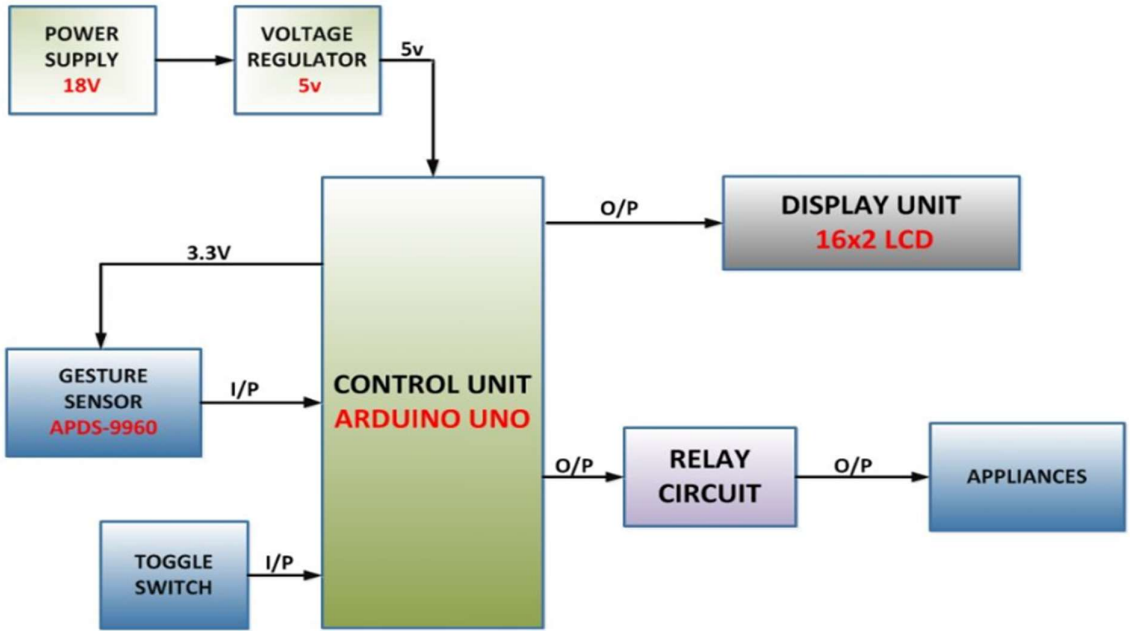


Figure 1.5 Block diagram of the system

The gesture-based home automation system begins functioning once powered, with the Arduino initializing essential libraries like Wire.h, SparkFun_APDS9960.h, and LiquidCrystal.h. These libraries enable I2C communication, gesture detection, and LCD display operations. The SparkFun library is used to handle the APDS-9960 gesture sensor, which recognizes directional finger movements. Initially, all appliances are turned OFF by sending LOW logic signals to the relay pins connected to Arduino. A tactile switch must be pressed to activate the system, after which a horizontal list of appliances appears on the LCD screen. Users can swipe left or right over the sensor to scroll through the appliances. A swipe upward turns OFF the selected device, and a swipe downward turns it ON by changing the corresponding pin's output logic.

The proximity sensing feature allows users to control all appliances at once—bringing a finger close turns all devices ON, while moving it away turns them OFF. Gesture detection is managed using the readGesture() method, with variables i, j, and k tracking movement directions. The loop() function handles user inputs and gestures, calling handleGesture() to update gesture variables. These guide the controlAppl() function in identifying and toggling device status. Devices are connected to Arduino pins A0–A3, with their states stored in dedicated variables. The LCD provides real-time feedback for each gesture.

1.2.6 Prathmesh Palandurkar el.al [2020] developed a gesture-controlled home automation system aimed particularly at aiding elderly and bedridden individuals by allowing them to operate household appliances with minimal physical effort. Their system uses a real-time vision-based interface where hand gestures are detected through a camera module connected to a Raspberry Pi. The system is programmed in Python using the OpenCV library for gesture recognition and operates appliances via a relay module based on Raspberry Pi's output. The primary aim is to establish seamless communication between humans and computerized systems for home control. The paper explores applications such as lighting, TV control, energy management, and security integration, highlighting both technical implementations and usability benefits. It also emphasizes the importance of addressing privacy and security challenges, advocating for the adoption of gesture-based automation as part of smart home infrastructure. The authors also discuss potential future improvements in gesture accuracy and the integration of AI for more advanced home automation. The study envisions the widespread use of this technology to make homes more accessible and intelligent, improving the quality of life for users with mobility limitations. [6]

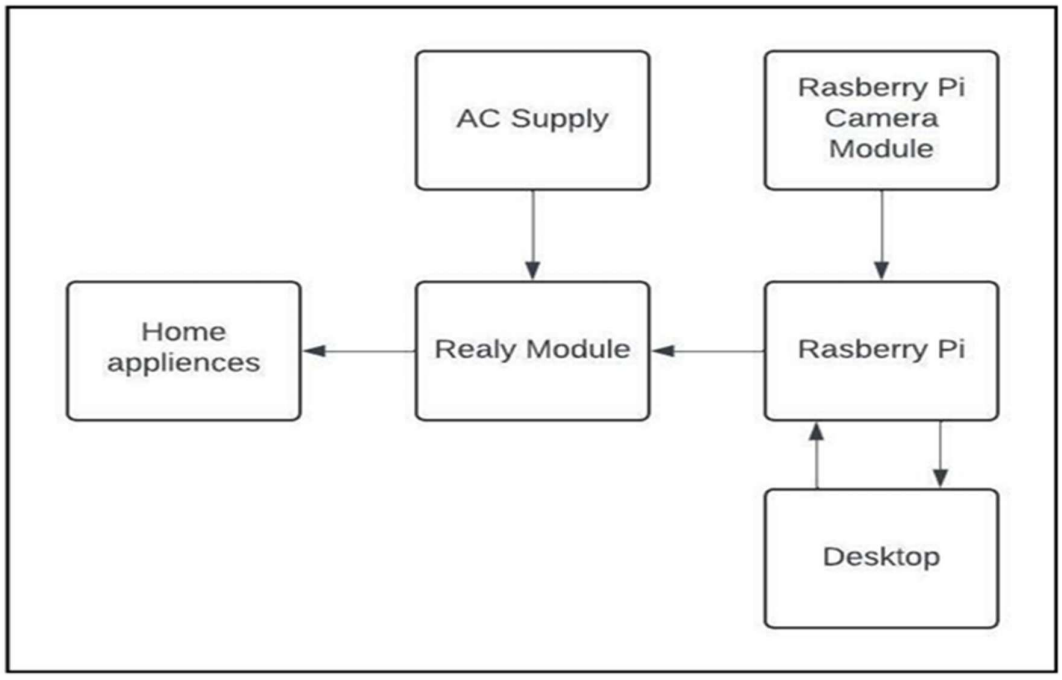


Figure 1.6: Block Diagram of the System

1.3 PROBLEM STATEMENT

The increasing frequency of contact with switches in homes and public spaces heightens the risk of bacterial and viral transmission, especially in high-touch areas. Traditional methods of controlling home appliances through physical switches or buttons contribute to this issue, creating opportunities for contamination. The need for a solution that reduces physical contact while maintaining ease of use is essential in promoting safer living conditions. Therefore, this project proposes an IoT-enabled gesture-controlled home automation system, offering a touch-free and hygienic alternative to conventional appliance control methods.

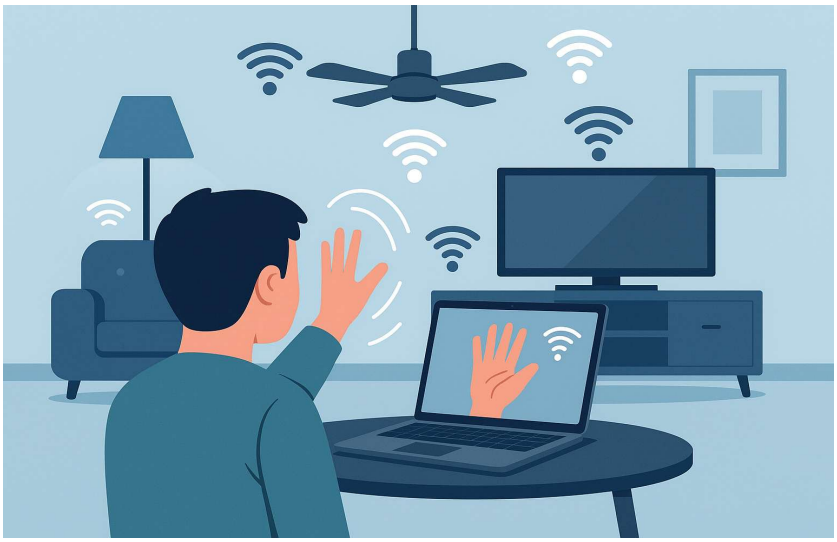


Figure 1.8 Solar Power Plants

1.1 OBJECTIVES:

- Create a touch-free system to control at least three appliances using hand gestures.
- Ensure quick and accurate gesture detection for seamless appliance control.
- Use IoT technology to connect the system to home appliances, allowing a laptop camera to detect hand gestures and control the appliances remotely, ensuring touchless operation.

CHAPTER 2
PROPOSED SYSTEM

2.1 BLOCK DIAGRAM

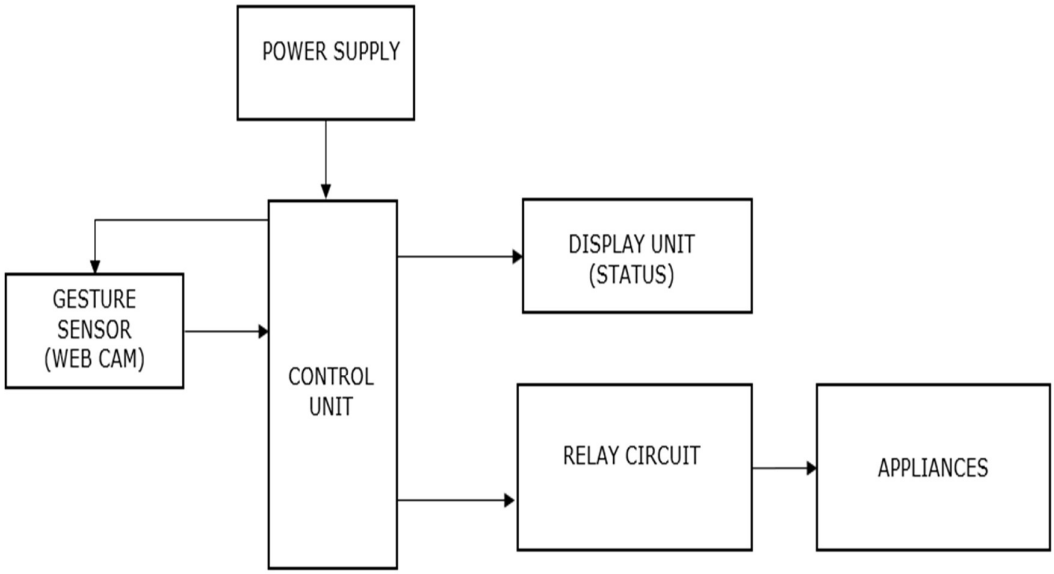


Figure 2.1 Block diagram of the system

The Internet of Things (IoT)-based solar power monitoring system is intended to continuously track and show the amount of electricity produced and light intensity of a solar panel. The solar panel's temperature, voltage, and current are measured by sensors in the system, and the data is wirelessly transmitted to data visualization interfaces such computer screens, mobile applications, and LCD displays. Three components make up the system's operation: sensing, processing, and visualization.

All in all, the system takes continuous measurements of the solar panel's properties, interprets the information, and produces alerts as well as real-time data visualization. By keeping an eye on the system's performance and making the required modifications, the user can improve performance and extend the solar panel's lifespan.

2.2 FLOWCHART

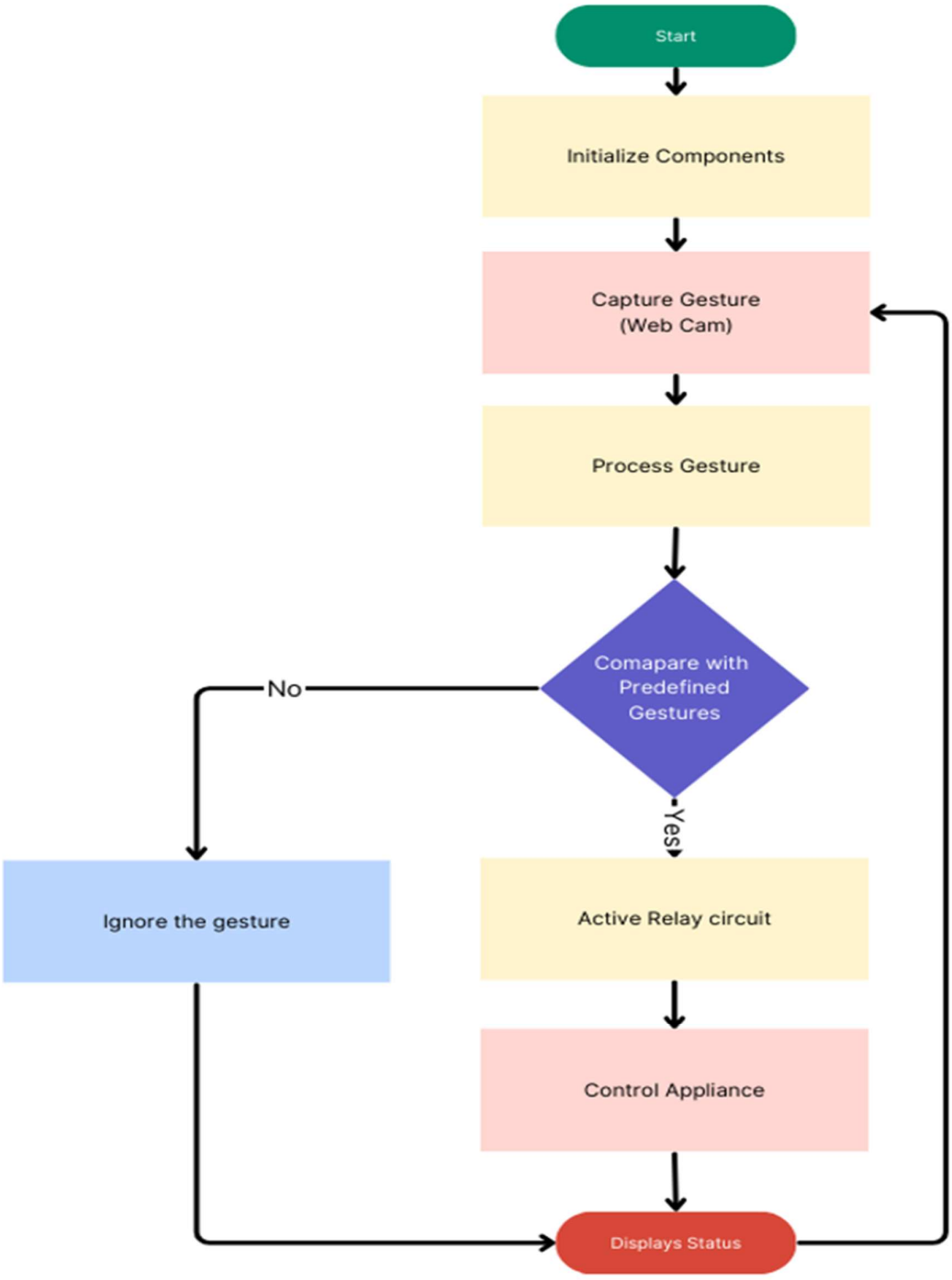


Figure 2.2 Flowchart of the System

2.3 SYSTEM SPECIFICATIONS

2.3.1 ARDUINO UNO:

Built around the Micro Chip ATmega328P microprocessor as shown in Figure, the Arduino Uno is a publicly available microcontroller device. Several expansion boards (shields) andother circuitry can be interfaced with the board's digital and analog input/output (I/O) pins. It can befueled by a USB connection or an external 9-volt battery and can operate on voltages between 7 and20 volts.

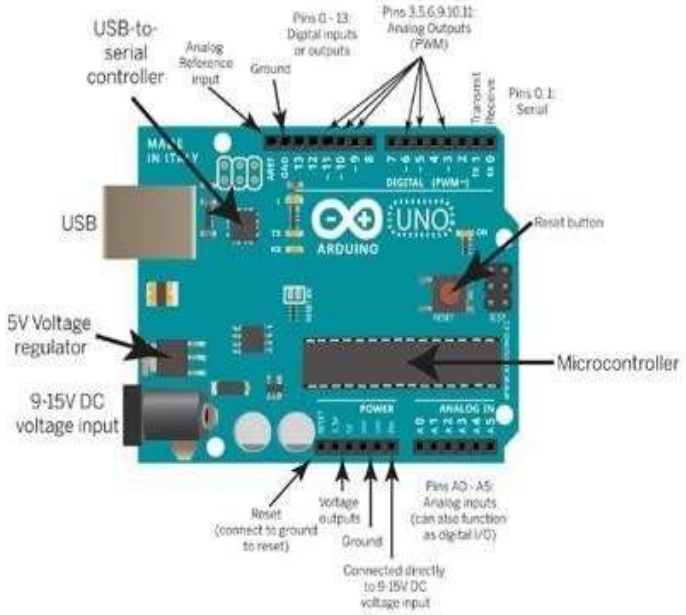


Figure 2.4 Arduino Uno

It is reasonably priced. It includes an open source hardware feature that lets people design their own kit by using an existing one as a guide. Windows, Linux, Macintosh, and other operating systems are all compatible with the Arduino software.

Additionally, it has an open-source software feature that allows skilled programmers to extend and modify the Arduino code in order to integrate it with pre-existing programming language libraries.

2.3.2 CURRENT SENSOR:

A relay is an electrically operated switch that allows control of a circuit by using a low-power signal to switch a higher-power circuit on or off. It consists of an electromagnet, a set of contacts, and a spring-loaded armature. When current flows through the electromagnet, it generates a magnetic field that pulls the armature, causing the contacts to either open or close, depending on the relay's configuration. Relays are widely used in automation systems, including IoT applications, for controlling devices such as motors, lights, and home appliances. They provide electrical isolation between control and load circuits, ensuring safety and preventing damage to sensitive components.

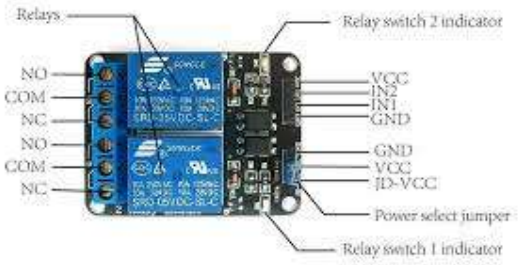


Figure 2.5 Relay sensor

In the "Gesture Driven IoT Smart Home System" project, relays are used to control the state (on/off) of the connected lights. When a gesture is detected, the system sends a signal to trigger the relay to switch the respective light on or off. The relay acts as an intermediary between the low-power control circuit and the high-power load (light bulbs), allowing the system to safely control the electrical devices without directly handling high currents.

2.3.2.1 APPLICATIONS:

- Home Automation: Control appliances with low-power signals.
- Industrial Control: Manage factory machines.
- Automotive: Control car lights and AC.
- Security: Activate alarms and locks.

2.3.3 ESP32-CAM module :

The ESP32-CAM module is a low-cost development board with a built-in camera, ideal for IoT projects. It features a 2MP camera, supporting video and image capture, and can be integrated with Wi-Fi for real-time streaming or remote control. The module is powered by the ESP32 chip, offering high performance and Bluetooth capabilities. It supports various applications such as surveillance, face recognition, and home automation. The compact size and versatility of the ESP32-CAM make it a popular choice for DIY and commercial IoT solutions.



Figure 2.6 ESP32-CAM module

The ESP32-CAM is a small yet powerful camera module with integrated Wi-Fi and Bluetooth capabilities. It supports video streaming and image capturing, making it ideal for surveillance and monitoring systems. The module uses the popular ESP32 chip, offering a cost-effective solution for IoT applications. With its compact design, it can easily be embedded into various smart devices and automation systems. The ESP32-CAM is widely used for facial recognition, object detection, and other image processing tasks in both personal and professional projects.

2.3.4.1 ADVANTAGES:

• Cost-Effective: Affordable for both hobbyists and professionals.

• Wireless Connectivity: Built-in Wi-Fi and Bluetooth for remote access and control.

• Compact Design: Small size, making it easy to integrate into various projects and devices

2.3.4 APPLIANCE(Bulb):

In a smart home system, bulbs are primarily used for general lighting to illuminate rooms efficiently. They can be controlled through gestures or automated schedules, offering convenience and energy savings. Smart bulbs also help create mood lighting by adjusting brightness and color. Additionally, they act as visual indicators for system states or alerts. Their integration enhances both functionality and aesthetics in modern living spaces. Bulbs also improve security by simulating presence when users are away. They support remote access, allowing control from smartphones or voice assistants.

2.3.4.1 ADVANTAGES:

- Energy-efficient with adjustable brightness settings.
- Enhances safety by providing instant lighting when needed.
- Can be controlled remotely via apps or voice assistants, Supports automation and scheduling for convenience.

CHAPTER 3

SOFTWARE IMPLEMENTATION

3.1 SIMULATION CIRCUIT

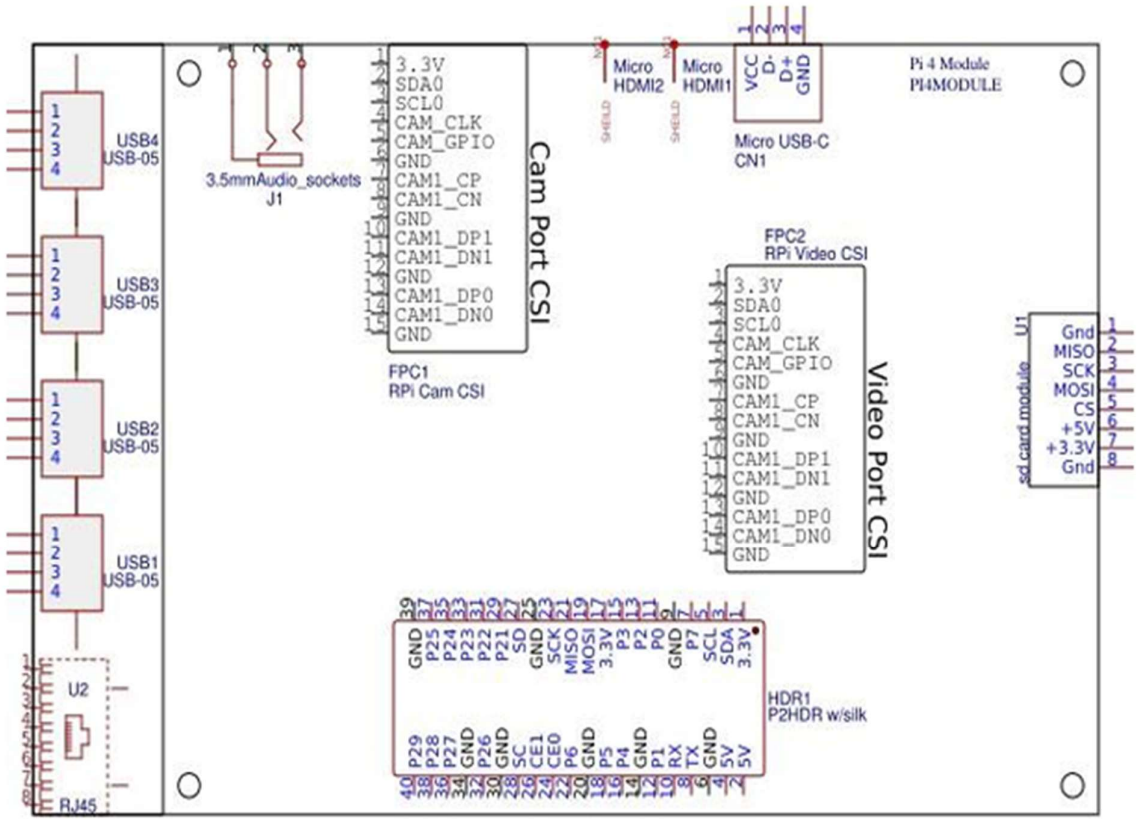


Figure 3.1 Simulation Circuit

shows the simulation circuit which works on the following flow. Hyperlinks have been given. The step-up converter, which is linked to the bridge rectifiers and employed to transform the AC voltage into DC voltage, receives the AC voltage as the energy source. The ripple in the converted DC voltage is mitigated using capacitor and the final DC voltage is given to the battery. The power supply of +5 volt is given to the Arduino in which the six pins of Arduino is interfaced with LCD and four pins are interfaced with the sensors. The relay and the transistor are used to get the output. When the power supply is given it will work as per the (Figure 3.1).

3.2 SIMULATION OUTPUT

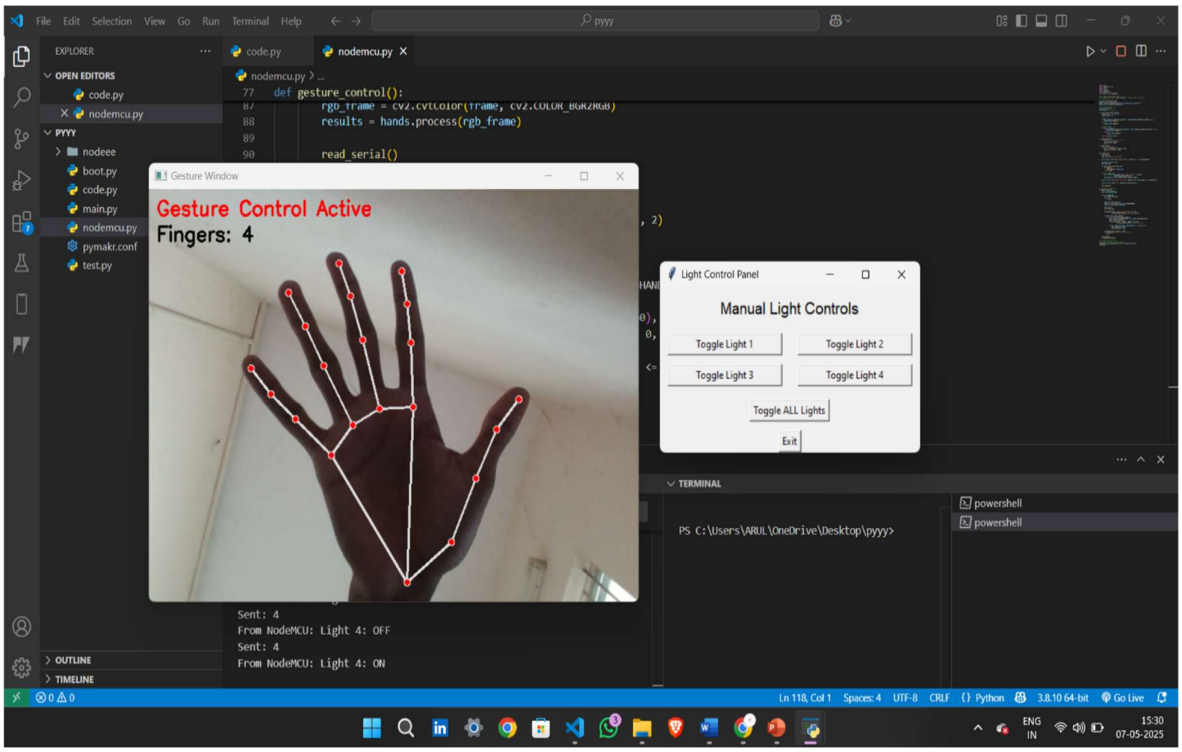


Figure 3.2 Simulation output diagram

This project integrates an IoT-based solar monitoring system with a real-time web dashboard using HTML, CSS, and JavaScript, merged with Arduino code. The implementation involves deploying sensors to measure solar parameters such as voltage, current, and temperature, interfaced with an Arduino microcontroller. These sensors transmit data to the Arduino, which processes and sends the information to a web server via a Wi-Fi module. The web dashboard is designed with HTML for structure, CSS for styling, and JavaScript for dynamic content updates. The JavaScript code uses AJAX to fetch real-time data from the server, displaying it on the dashboard in an intuitive format using charts and gauges for easy monitoring. This integration provides seamless and efficient monitoring, offering real-time insights into solar panel performance, enhancing data visualization, and ensuring continuous tracking of solar energy metrics for better system management and optimization.

3.3 Types of Sensors needed for Gesture Driven IoT Smart Home System

In a Gesture Driven IoT Smart Home System, vision-based sensors play a crucial role in detecting and interpreting hand gestures. One of the most widely used components for this purpose is a camera module such as the ESP32-CAM. This module captures real-time video input of hand movements, which is then processed by computer vision algorithms to identify specific gestures. These gestures are mapped to control various home appliances like lights and fans.

In addition to vision sensors, motion detection sensors such as accelerometers or gyroscopes can also be employed in wearable gesture systems. These sensors detect changes in hand orientation, speed, and direction, making them suitable for recognizing dynamic gestures. While not essential in a camera-based system, they are beneficial in environments where visual tracking may be obstructed.

Furthermore, infrared (IR) sensors or ultrasonic sensors can be used to detect the presence and distance of a hand from the sensor. These can add a layer of proximity-based interaction, allowing devices to respond when a hand is waved nearby. Combining different types of sensors enhances the accuracy, flexibility, and reliability of the gesture recognition system in a smart home environment.

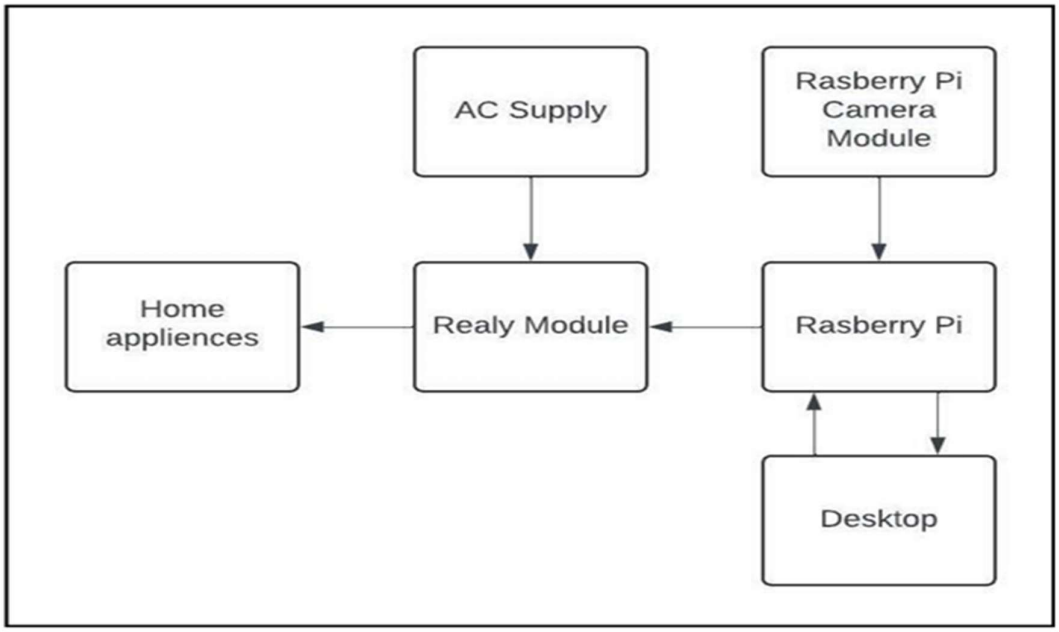


Figure 3.3 Types of Sensors needed for IoT based Solar Power Monitoring System

3.4 ALGORITHM

1. Start the system and initialize all components:

- Start serial communication between the PC and NodeMCU.
- Initialize camera for video capture.
- Initialize MediaPipe hand tracking module.
- Initialize digital pins for all connected relays (lights) on the NodeMCU.

2. Capture and Process Video Input:

- Continuously capture frames from the webcam.
- Convert the captured frame to RGB and process it using the MediaPipe Hand model.
- Detect hand landmarks and count the number of extended fingers.

3. Determine Gesture:

- Use landmark positions to calculate how many fingers are raised.
- Match the finger count with a predefined command (e.g., 1 finger = toggle light 1, 2 fingers = toggle light 2, etc.).

4. Send Command to NodeMCU:

- If a valid gesture is detected and a cooldown period has passed, send the corresponding command (1–5) to the NodeMCU via serial.
- Display the detected gesture and the command sent on the screen.

5. NodeMCU Receives and Executes Command:

- Read the incoming serial data on NodeMCU.
- Based on the received command:
  - Toggle the corresponding relay state (ON/OFF) for the associated light.
  - For command '5', toggle all relays at once.
- Print feedback to serial for monitoring (e.g., "Light 1: ON").

6. Loop steps 2 to 5 continuously until the system is manually stopped.

7. End the process when the quit key is pressed or the window is closed.

CHAPTER 4

RESULT AND DISCUSSION

4.1 HARDWARE OUTPUT:

The Gesture Driven IoT Smart Home System was successfully developed and tested. It enables users to control multiple home lights using simple hand gestures, enhancing accessibility and convenience. The system demonstrated reliable real-time response through Media-Pipe and serial communication with NodeMCU. Manual control was also integrated via a graphical interface for added flexibility. Overall, the project proves the effectiveness of gesture-based control in smart home automation. The system also contributes to energy efficiency by allowing users to control lighting more intuitively and selectively.

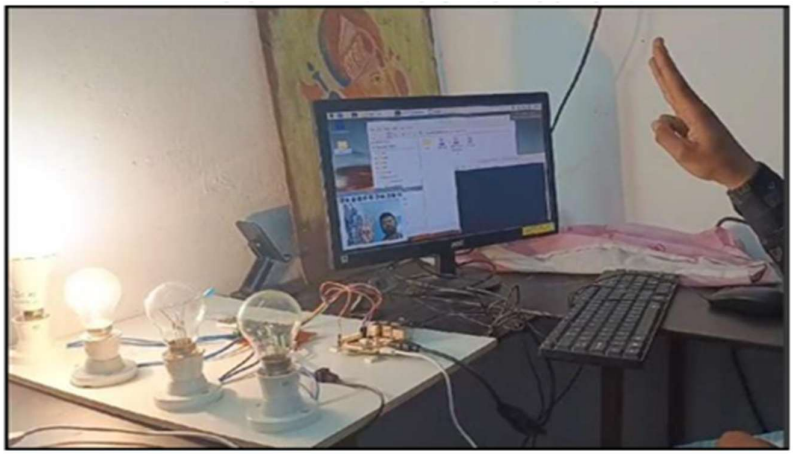


Figure 4.1 Solar Power Monitoring System

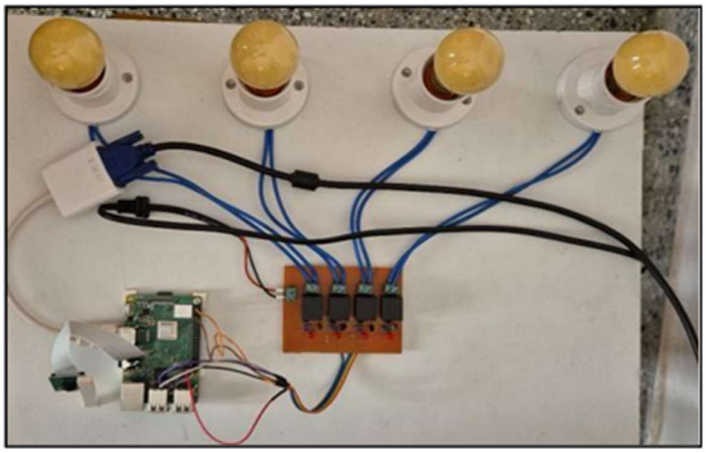


Figure 4.2 Hardware output

CHAPTER 5
CONCLUSION

The system offers numerous benefits such as effective energy administration, enhanced solar panel efficiency, lower maintenance expenses, and enhanced system dependability. It continuously oversees and refreshes data, enabling real-time monitoring of solar panel performance due to fluctuations in solar radiation over time and location. This proposed system records voltage and current parameters and regularly updates them with new values. Monitoring the solar panel allows for straightforward daily or monthly analysis. Additionally, the system can identify any errors in the data generated in case of uncertainty.

REFERENCES

[1]. Harale, A.D., & Bankar, A.S. Gestures Controlled Home Automation using Deep Learning. International Journal of Current Engineering and Technology, Vol. 11, No. 6.

[2]. Baleegha, F., & Mushtaq, B. IoT-based Smart Home Automation Using Gesture Control. IECE Transactions on Internet of Things, 2(4), 74–82.

[3]. Chinamaya, H.G., & Kumar, N.N. Gesture Based Smart Home Automation System Using Real Time Inputs. International Journal of Latest Research in Engineering and Technology, pp. 108–112.

[4]. Manikanta, S.P., & Shubhaker, B. Gesture Based Appliance Automation System Using IoT. Journal of Engineering Science, Vol. 11, Issue 4.

[5]. Yang, L.I., Huang, J., Feng, T.I.A.N., Wang, H.-A., & Dai, G.-Z. (2019). Gesture Interaction in Virtual Reality. Virtual Reality & Intelligent Hardware, 1(1), 84–112. [CrossRef]

[6]. Kheratkar, N., Bhavani, S., Jarali, A., Pathak, A., & Kumbhar, S. (2020, May). Gesture Controlled Home Automation using CNN. In 2020 4th International Conference on Intelligent Computing and Control Systems (ICICCS) (pp. 620–626). IEEE. [CrossRef]

[7]. Tomar, D., Nauni, D., Zaidi, A.M., & Kaur, M. (2023, November). Gesture-Controlled Home Automation for the Differently Abled: Enhanced Accessibility and Independence. In 2023 3rd International Conference on Technological Advancements in Computational Sciences (ICTACS) (pp. 1560–1564). IEEE. [CrossRef].

APPENDIX

ARDUINO CODE:

```
#include <ESP8266WiFi.h> // Include the WiFi library for ESP8266


// Define pin connections for each light
#define CEILING_LIGHT D1
#define DESK_LAMP D2
#define BEDSIDE_LIGHT D3
#define AMBIENT_LIGHT D4


// Boolean variables to store the current state of each light
bool ceilingState = false;
bool deskState = false;
bool bedsideState = false;
bool ambientState = false;


void setup() {
  Serial.begin(9600); // Initialize serial communication at 9600 baud rate


  // Set pin modes for all light pins
  pinMode(CEILING_LIGHT, OUTPUT);
  pinMode(DESK_LAMP, OUTPUT);
  pinMode(BEDSIDE_LIGHT, OUTPUT);
  pinMode(AMBIENT_LIGHT, OUTPUT);
```

```
  // Turn off all lights initially
  digitalWrite(CEILING_LIGHT, LOW);
  digitalWrite(DESK_LAMP, LOW);
  digitalWrite(BEDSIDE_LIGHT, LOW);
  digitalWrite(AMBIENT_LIGHT, LOW);

  Serial.println("NodeMCU Ready for Light Control"); // Print ready message to serial monitor
}

void loop() {
  if (Serial.available()) { // Check if data is available from the serial monitor
    char command = Serial.read(); // Read the command character

    switch (command) {
      case '1': // Toggle ceiling light
        ceilingState = !ceilingState;
        digitalWrite(CEILING_LIGHT, ceilingState ? HIGH : LOW);
        Serial.println(ceilingState ? "Light 1: ON" : "Light 1: OFF");
        break;
      case '2': // Toggle desk lamp
        deskState = !deskState;
        digitalWrite(DESK_LAMP, deskState ? HIGH : LOW);
        Serial.println(deskState ? "Light 2: ON" : "Light 2: OFF");
        break;
      case '3': // Toggle bedside light
```

```
            bedsideState = !bedsideState;
            digitalWrite(BEDSIDE_LIGHT, bedsideState ? HIGH : LOW);
            Serial.println(bedsideState ? "Light 3: ON" : "Light 3: OFF");
            break;
        case '4': // Toggle ambient light
            ambientState = !ambientState;
            digitalWrite(AMBIENT_LIGHT, ambientState ? HIGH : LOW);
            Serial.println(ambientState ? "Light 4: ON" : "Light 4: OFF");
            break;
        case '5': // Toggle all lights
            bool newState = !(ceilingState && deskState && bedsideState && ambientState);
            ceilingState = deskState = bedsideState = ambientState = newState;

            digitalWrite(CEILING_LIGHT, ceilingState ? HIGH : LOW);
            digitalWrite(DESK_LAMP, deskState ? HIGH : LOW);
            digitalWrite(BEDSIDE_LIGHT, bedsideState ? HIGH : LOW);
            digitalWrite(AMBIENT_LIGHT, ambientState ? HIGH : LOW);

            Serial.println(newState ? "All Lights: ON" : "All Lights: OFF");
            break;
      }
   }
}
```

CODE:

```
import cv2
import mediapipe as mp
import time
import serial
import threading
import tkinter as tk
from tkinter import messagebox

# ===== Serial Communication =====
ser = serial.Serial('COM7', 9600, timeout=1)  # Change 'COM7' to your port

# ===== MediaPipe Setup =====
mp_hands = mp.solutions.hands
hands = mp_hands.Hands(max_num_hands=1, min_detection_confidence=0.7)
mp_drawing = mp.solutions.drawing_utils

# ===== Global States =====
last_action_time = 0
action_delay = 5  # seconds

def count_fingers(hand_landmarks):
    tip_ids = [4, 8, 12, 16, 20]
    finger_states = []

    # Thumb
    if hand_landmarks.landmark[tip_ids[0]].x < hand_landmarks.landmark[tip_ids[0] - 1].x:
        finger_states.append(1)
    else:
        finger_states.append(0)

    # Other fingers
    for id in range(1, 5):
```

```
        if hand_landmarks.landmark[tip_ids[id]].y < hand_landmarks.landmark[tip_ids[id] - 2].y:
            finger_states.append(1)
        else:
            finger_states.append(0)

    return sum(finger_states)

def send_command(cmd):
    if cmd in ['1', '2', '3', '4', '5']:
        ser.write(cmd.encode())
        print(f"Sent: {cmd}")

def read_serial():
    while ser.in_waiting > 0:
        msg = ser.readline().decode().strip()
        print("From NodeMCU:", msg)

# ===== GUI =====
def create_gui():
    root = tk.Tk()
    root.title("Light Control Panel")

    tk.Label(root, text="Manual Light Controls", font=("Arial", 14)).pack(pady=10)

    btn_frame = tk.Frame(root)
    btn_frame.pack()

    def make_toggle_button(light_num):
        def toggle():
            send_command(str(light_num))
        return toggle

    for i in range(1, 5):
        btn = tk.Button(btn_frame, text=f"Toggle Light {i}", width=20,
```

```
                    command=make_toggle_button(i))
        btn.grid(row=(i-1)//2, column=(i-1)%2, padx=10, pady=5)

    tk.Button(root, text="Toggle ALL Lights", command=lambda:
send_command('5')).pack(pady=10)

    tk.Button(root, text="Exit", command=root.destroy).pack()

    root.mainloop()

# ===== Gesture Thread =====
def gesture_control():
    global last_action_time
    cap = cv2.VideoCapture(0)

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            continue

        frame = cv2.flip(frame, 1)
        rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = hands.process(rgb_frame)

        read_serial()
        now = time.time()

        cv2.putText(frame, "Gesture Control Active", (10, 30),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)

        if results.multi_hand_landmarks:
            for lm in results.multi_hand_landmarks:
                mp_drawing.draw_landmarks(frame, lm, mp_hands.HAND_CONNECTIONS)
                count = count_fingers(lm)
```

```
            cv2.putText(frame, f"Fingers: {count}", (10, 60),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 0), 2)

            if now - last_action_time > action_delay and 1 <= count <= 5:
                send_command(str(count))
                last_action_time = now

        cv2.imshow("Gesture Window", frame)
        if cv2.waitKey(1) & 0xFF == 27:
            break

    cap.release()
    cv2.destroyAllWindows()

# ===== Main Execution =====
# Run gesture and GUI in parallel threads
threading.Thread(target=gesture_control, daemon=True).start()
create_gui()
```